WEB DEVELOPMENT · 3RD SEMESTER

Week 03 · Block 3b

Cascade & Box Model

<web-development> · Hochschule Ansbach
Matthias Kellnhofer

After this block, you can…

- Predict which rule **wins** a conflict, using **specificity** and **source order**
- Read a selector's specificity at a glance
- Know which properties **inherit** — and which don't
- Explain the **box model**: content, padding, border, margin
- Use `box-sizing: border-box` for predictable sizing

In Block 3a you wrote a `.card` rule and a `#about` rule for the same section.

Which one wins?

Today: how CSS resolves every conflict — and how every element is sized.

PART 1

Specificity & the Cascade

Many rules, one element

```
p        { color: black; }
.lead    { color: gray;  }
#intro   { color: navy;  }
```

All three can target the same paragraph. CSS picks a winner with the **cascade** — in this order:

1. **Specificity** — how specific is the selector?
2. **Source order** — if still tied, the **last** rule wins
3. (`!important` *can override even that — more soon)*

How specific is a selector?

From least to most specific:

- **type** — `p`, `h1` *(least)*
- **class** — `.card`, `.note`
- **id** — `#about`
- **inline** `style="…"` — *beats them all*

Count each kind as **(ids, classes, types)**, then compare left to right — a higher number in an earlier slot wins.

Count it up

selector	(id, class, type)
p	(0, 0, 1)
nav a	(0, 0, 2)
.lead	(0, 1, 0)
.card p	(0, 1, 1)
#about	(1, 0, 0)

Compare **ids first**, then classes, then types. One `#id` (1,0,0) beats `.card p` (0,1,1) — a single id outranks any number of classes.

The Block 3a cliffhanger, solved

```
.card   { background-color: white; }   /* (0, 1, 0)          */
#about  { background-color: #eef;  }   /* (1, 0, 0)  ← wins   */
```

`#about` is more specific, so the *About* card is blue. Mystery solved.

This is exactly why we style with **classes**, not ids: an id is hard to override later. Keep specificity *low and even*, and life is easy.

Let the browser tell you

No need to compute it in your head:

- Dev tools → **Elements** → the **Styles** pane
- The winning declaration is shown; overridden ones are ~~**struck through**~~
- Click any value to see which rule set it

When a style "isn't working", it's usually being **overridden** — the Styles pane shows you by what.

PART 2

Source Order & !important

Ties break by order — and the escape hatch

```
.btn { color: blue; }
.btn { color: red;  }   /* same specificity → this one wins */
```

- **Equal specificity → the later rule wins.** This is why your overrides go *after* a library's CSS.
- `!important` (`color: red !important;`) overrides **everything** — specificity, order, all of it.

Treat `!important` as a last resort. It wins the battle but breaks the cascade's natural order — and the only way to beat one `!important` is another. Avoid the arms race.

PART 3

Inheritance

Some properties flow down the tree

```
body {
  color: #333;
  font-family: system-ui, sans-serif;
}
```

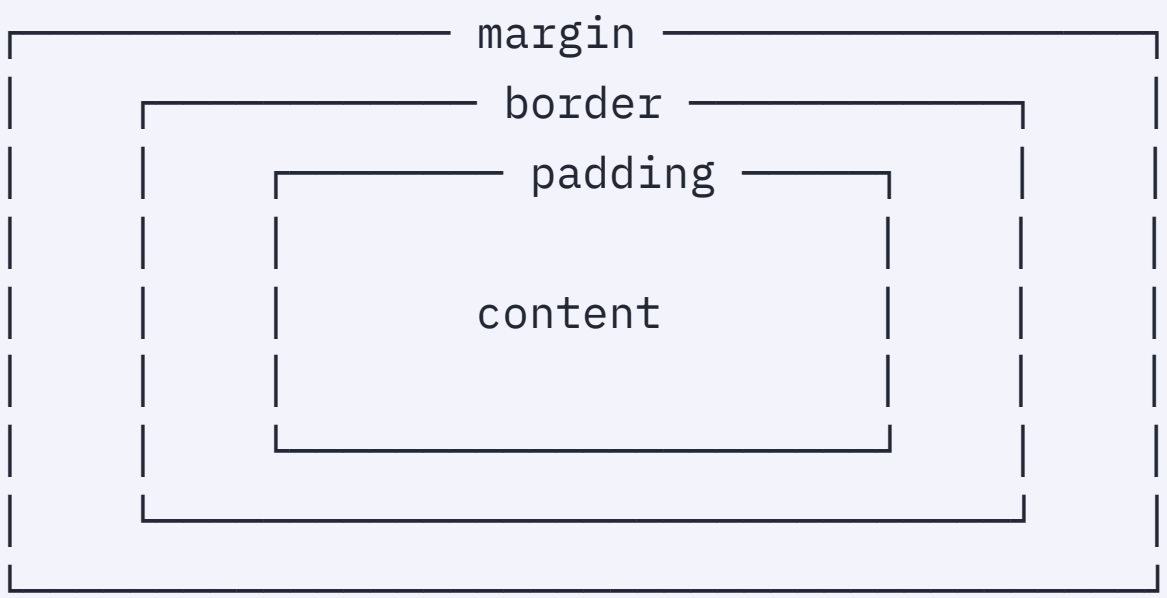
- **Text** properties **inherit** — `color`, `font-*`, `line-height`, `text-align`. Set them once on `body`; every child gets them for free.
- **Box** properties **don't** — `background`, `border`, `margin`, `padding`, `width`. Each element stands alone.

Rule of thumb: if it describes the **text**, it probably inherits. If it describes the **box**, it doesn't.

PART 4

The Box Model

Every element is a box



Four layers, inside out: **content** → **padding** → **border** → **margin**.

What each layer does

```
.card {
  padding: 20px;              /* space INSIDE the border  */
  border: 1px solid #ccc;     /* the edge itself          */
  margin: 16px;               /* space OUTSIDE, between boxes */
}
```

- **content** — the text or image itself
- **padding** — breathing room between content and border
- **border** — the visible edge
- **margin** — the gap that pushes neighbouring boxes away

(We'll use these for real spacing & layout in Block 3c.)

PART 5

box-sizing

The width gotcha

```
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid;
}
```

How wide is `.box` on screen?

- By default (**`content-box`**), `width` sizes only the **content**
- Real width = `200 + 20 + 20 + 5 + 5` = **250px** 😱
- Padding and border are added *on top* — surprising every time

border-box to the rescue

```
* {
  box-sizing: border-box;
}
```

- Now `width: 200px` means the box is **200px** — padding and border **included**
- Predictable sizing; the maths just works
- Set it once with the universal `*` selector — almost every project starts here

Recap

- The **cascade** resolves conflicts: **specificity**, then **source order** (`!important` on top — avoid)
- Specificity ranks **inline > id > class > type**; count `(id, class, type)`
- **Text** properties **inherit**; **box** properties don't
- Every element is a **box**: content → padding → border → margin
- **`box-sizing: border-box`** makes `width` predictable — set it globally

<EXERCISE> · 45 MIN

Your turn 👉

Make CSS conflicts **predictable**, then take the box model apart.

1. Create three rules for one element (`type` , `.class` , `#id`) — **predict** the winner, then check
2. Use the **Styles pane** to see overridden rules struck through
3. Give a box **padding**, **border**, **margin** — watch each layer in the box-model diagram
4. Flip **`box-sizing`** between `content-box` and `border-box` — measure the difference

Starter: `exercises/week03/3b/` — open `README.md` . The page has ready-made conflicts and a box to dissect.